Multimodal, Explainable Skin-Lesion Classification

A research-grade upgrade of a three-stage skin-lesion classifier: **image + patient-metadata fusion**, **principled class-imbalance handling** with a **diffusion-augmentation ablation**, **Grad-CAM explainability**, and **medical-grade evaluation** (sensitivity/specificity, ROC-AUC, calibration, McNemar significance) — packaged as Colab notebooks, an importable `src/` library with unit tests, an explainable Streamlit demo, and a paper-style report.

> **Medical disclaimer.** This is a research prototype and educational demo, **not a diagnostic device**. Do not use it for real clinical decisions. See `report/report.md` §9.

What this is

The original project trained three image-only EfficientNetB3 CNNs on class-balanced (undersampled) data and reported accuracy. This version keeps the clinically-sensible hierarchy but turns it into a defensible research artifact:

```
Stage 1 : benign vs malignant            (all 7 HAM10000 dx)
   ├─ benign    → Stage 2a : {nv, bkl, vasc}
   └─ malignant → Stage 2b : {mel, bcc, akiec}
```

Upgrade	Where
Multimodal image + metadata (age, sex, site) late fusion at every stage	`src/model.py`, `src/metadata.py`
Full-data training + class weights (no undersampling) + live augmentation	`src/data.py`
Diffusion (DDPM) augmentation of the rarest malignancy (`akiec`) + ablation	`src/diffusion.py`, `notebooks/05`
Grad-CAM explanations	`src/gradcam.py`
Sensitivity/specificity, ROC-AUC, ECE, confusion, McNemar	`src/evaluate.py`, `notebooks/06`
Leakage-free global split shared across all stages	`src/data.assign_global_split`
End-to-end multimodal inference for the app	`src/pipeline.py`

The original notebooks are preserved in `archive/` as the image-only comparison point.

Repo structure

```
├── notebooks/                  # Colab-ready, run 01 → 06 in order
│   ├── 01_data_prep_ham10000.ipynb          # download HAM, global split, fit
```

```
metadata
│   ├── 02_stage1_benign_malignant_multimodal.ipynb
│   ├── 03_stage2a_benign_type_multimodal.ipynb
│   ├── 04_stage2b_malignant_type_multimodal.ipynb
│   ├── 05_diffusion_rare_class.ipynb        # train DDPM, synthesize akiec
│   └── 06_evaluation_figures_gradcam.ipynb  # all metrics/figures + results.json
├── src/                        # importable, CPU-unit-testable
│   ├── data.py  metadata.py  model.py  gradcam.py  diffusion.py  evaluate.py
pipeline.py
├── tests/                      # pytest suite (see "Testing")
├── report/
│   ├── report.md               # paper-style writeup (placeholders)
│   ├── fill_report.py          # results.json → report_final.md
│   └── figures/                # PNGs emitted by notebook 06
├── app.py  components/  utils/  static/  assets/   # Streamlit demo
├── models/                     # trained .keras + preprocessing.json (gitignored)
├── archive/                    # original image-only notebooks
├── requirements.txt            # app runtime
└── requirements-train.txt      # + training/diffusion/eval deps
```

Reproduce the models (Google Colab, free T4)

The notebooks are written for Colab (they `!pip` what they need, download HAM10000 from Kaggle, and save artifacts to Google Drive or the local session).

1. **`01_data_prep_ham10000.ipynb`** — downloads HAM10000 (`kmader/skin-cancer-mnist-ham10000`; needs a Kaggle token), builds the global stratified split → `models/ham_split.csv`, and fits the `MetadataProcessor` (train split only) → `models/preprocessing.json`.
2. **`02`–`04`** — train the image-only **baseline** and the **multimodal** model for each stage under balanced class weights + live augmentation. Saves `{stage}_baseline.keras` and `{stage}_multimodal.keras`. ~15–25 min/stage on a T4.
3. **`05_diffusion_rare_class.ipynb`** — trains a compact 64×64 DDPM on `akiec`, generates synthetic images, and retrains Stage-2b to produce `malignant_multimodal_aug.keras` for the ablation. Time-boxed; see the diffusers/hub note in `requirements-train.txt`.
4. **`06_evaluation_figures_gradcam.ipynb`** — evaluates every stage and the end-to-end pipeline on the held-out split, writing `report/figures/*.png`, `report/figures/summary.md`, and `report/results.json`.

Then populate the report locally:

```
python report/fill_report.py    # report/results.json → report/report_final.md
```

Copy the trained `models/*.keras` + `preprocessing.json` into `models/` to run the app locally.

Run the demo app

```
pip install -r requirements.txt
streamlit run app.py
```

Upload a dermatoscopic image and (optionally) enter age / sex / body site. The app loads the local pipeline from `models/`, shows the staged prediction with confidences and a **Grad-CAM overlay**, and degrades gracefully (stage-1 only) if a sub-stage model is missing. If no local models are present it can fall back to a remote HuggingFace API when `.env` provides `API_URL` / `HF_API_TOKEN`.

Testing

CPU-only unit tests cover every `src/` module (metadata round-trip, data pipeline + splits, model shapes, Grad-CAM, evaluation metrics, end-to-end pipeline routing, and the report filler). The diffusion tests self-skip if `torch` / `diffusers` aren't installed.

```
pip install -r requirements-train.txt   # brings in pytest deps + torch/diffusers
pytest                                   # from the project root
```

The suite builds a tiny synthetic HAM-shaped dataset on the fly — no download or GPU needed.

Report

The full writeup — abstract, methods, results tables (baseline vs. multimodal, imbalance and diffusion ablations), ROC/confusion/calibration/Grad-CAM figures, discussion, limitations, and ethics — is in `report/report.md`. Numbers are placeholders until you run notebook 06 and `report/fill_report.py`.

Authors

- Iza KC
- Prajwal Khatiwada
- Samrat Adhikari
- Er. Pralhad Chapagain

Data & license

Uses the **HAM10000** dataset (Tschandl et al., 2018) under its dataset terms. No patient-identifiable data is introduced. See the report's References for full citations.